

# The power of differentiation

Jonathon Hare

## Topics

- The big idea: optimisation by following gradients
- Recap: what are gradients and how do we find them?
- Jacobians as local linear maps
- Jacobian-vector and vector-Jacobian products

## The big idea: optimisation by following gradients

- Fundamentally, we're interested in machines that we train by optimising parameters
  - How do we select those parameters?
- In deep learning/differentiable programming we typically define an objective function that we *minimise* (or *maximise*) with respect to those parameters
- This implies that we're looking for points at which the gradient of the objective function is zero w.r.t the parameters

## The big idea: optimisation by following gradients

- Gradient based optimisation is a *big* field!
  - First order methods, second order methods, subgradient methods...
- With deep learning we're primarily interested in first-order methods<sup>1</sup>.
  - Primarily using variants of gradient descent: a function  $F(\mathbf{x})$  has a minima<sup>2</sup> (or a saddle-point) at a point  $\mathbf{x} = \mathbf{a}$  where  $\mathbf{a}$  is given by applying  $\mathbf{a}_{n+1} = \mathbf{a}_n - \alpha \nabla F(\mathbf{a}_n)$  until convergence from some initial point  $\mathbf{a}_0$ .

## Recap: what are gradients and how do we find them?

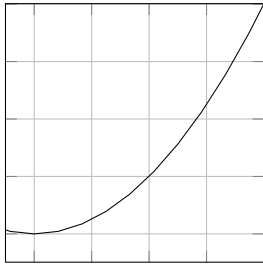
### The derivative in 1D

- Recall that the gradient of a straight line is  $\frac{\Delta y}{\Delta x}$ .
- For an arbitrary real-valued function,  $f(a)$ , we can approximate the derivative,  $f'(a)$  using the gradient of the *secant line* defined by  $(a, f(a))$  and a point a small distance,  $h$ , away  $(a + h, f(a + h))$ :  $f'(a) \approx \frac{f(a+h)-f(a)}{h}$ .
  - This expression is 'Newton's Quotient' or 'Fermat's Difference Quotient'.
  - As  $h$  becomes smaller, the approximated derivative becomes more accurate.
  - If we take the limit as  $h \rightarrow 0$ , then we have an exact expression for the derivative:  $\frac{df}{da} = f'(a) = \lim_{h \rightarrow 0} \frac{f(a+h)-f(a)}{h}$ .

---

<sup>1</sup>Second order gradient optimisers are potentially better, but for systems with many variables are currently impractical as they require computing the Hessian.

<sup>2</sup>not necessarily global or unique



### Recap: what are gradients and how do we find them?

*The derivative of  $y = x^2$  from first principles*

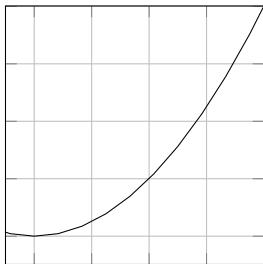
$$\begin{aligned}
 y &= x^2 \\
 \frac{dy}{dx} &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\
 \frac{dy}{dx} &= \lim_{h \rightarrow 0} \frac{x^2 + h^2 + 2hx - x^2}{h} \\
 \frac{dy}{dx} &= \lim_{h \rightarrow 0} \frac{h^2 + 2hx}{h} \\
 \frac{dy}{dx} &= \lim_{h \rightarrow 0} (h + 2x) \\
 \frac{dy}{dx} &= 2x
 \end{aligned}$$

### Recap: what are gradients and how do we find them?

*Aside: numerical approximation of the derivative*

- For numerical computation of derivatives it is better to use a “centralised” definition of the derivative:

- $f'(a) = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a-h)}{2h}$
- The bit inside the limit is known as the *symmetric difference quotient*
- For small values of  $h$  this has less error than the standard one-sided difference quotient.



### Recap: what are gradients and how do we find them?

*Aside: numerical approximation of the derivative*

- If you are going to use difference quotients to estimate derivatives you need to be aware of potential rounding errors due to floating point representations.
  - Calculating derivatives this way using less than 64-bit precision is rarely going to be useful. (Numbers are not represented exactly, so even if  $h$  is represented exactly,  $x + h$  will probably not be)
  - You need to pick an appropriate  $h$  - too small and the subtraction will have a large rounding error!

## Recap: what are gradients and how do we find them?

### Derivatives of deeper functions

- Deep learning is all about optimising deeper functions; functions that are compositions of other functions
  - e.g.  $z = f \circ g(x) = f(g(x))$
- The chain rule of calculus tells us how to differentiate compositions of functions:
  - $\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx}$

## Recap: what are gradients and how do we find them?

Example: differentiating  $z = x^4$

Note that this is a silly example that just serves to demonstrate the principle!

$$\begin{aligned} z &= x^4 \\ z &= (x^2)^2 = y^2 \quad \text{where } y = x^2 \\ \frac{dz}{dx} &= \frac{dz}{dy} \frac{dy}{dx} = (2y)(2x) = (2x^2)(2x) = 4x^3 \end{aligned}$$

Equivalently, from first principles:

$$\begin{aligned} z &= x^4 \\ \frac{dz}{dx} &= \lim_{h \rightarrow 0} \frac{(x+h)^4 - x^4}{h} \\ \frac{dz}{dx} &= \lim_{h \rightarrow 0} \frac{h^4 + 4h^3x + 6h^2x^2 + 4hx^3 + x^4 - x^4}{h} \\ \frac{dz}{dx} &= \lim_{h \rightarrow 0} h^3 + 4h^2x + 6hx^2 + 4x^3 = 4x^3 \end{aligned}$$

## Recap: what are gradients and how do we find them?

### Vector functions

- What if we're dealing with a *vector* function,  $\mathbf{y}(t)$ ?
  - This can be split into its constituent coordinate functions:  $\mathbf{y}(t) = (y_1(t), \dots, y_n(t))$ .
  - Thus the derivative is a vector (the 'tangent vector'),  $\mathbf{y}'(t) = (y'_1(t), \dots, y'_n(t))$ , which consists of the derivatives of the coordinate functions.
  - Equivalently,  $\mathbf{y}'(t) = \lim_{h \rightarrow 0} \frac{\mathbf{y}(t+h) - \mathbf{y}(t)}{h}$  if the limit exists.

## Recap: what are gradients and how do we find them?

### Functions of multiple variables: partial differentiation

- What if the function we're trying to deal with has multiple variables<sup>3</sup> (e.g.  $f(x, y) = x^2 + xy + y^2$ )?
  - This expression has a pair of *partial derivatives*,  $\frac{\partial f}{\partial x} = 2x + y$  and  $\frac{\partial f}{\partial y} = x + 2y$ , computed by differentiating with respect to each variable  $x$  and  $y$  whilst holding the other(s) constant.
- In general, the partial derivative of a function  $f(x_1, \dots, x_n)$  at a point  $(a_1, \dots, a_n)$  is given by:  $\frac{\partial f}{\partial x_i}(a_1, \dots, a_n) = \lim_{h \rightarrow 0} \frac{f(a_1, \dots, a_i+h, \dots, a_n) - f(a_1, \dots, a_i, \dots, a_n)}{h}$ .
- The vector of partial derivatives of a scalar-value multivariate function,  $f(x_1, \dots, x_n)$  at a point  $(a_1, \dots, a_n)$ , can be arranged into a vector:  $\nabla f(a_1, \dots, a_n) = \left( \frac{\partial f}{\partial x_1}(a_1, \dots, a_n), \dots, \frac{\partial f}{\partial x_n}(a_1, \dots, a_n) \right)$ .
  - This is the **gradient** of  $f$  at  $a$ .
- In the case of a vector-valued multivariate function, the partial derivatives form a matrix called the **Jacobian**.

---

<sup>3</sup>A multivariate function

## Recap: what are gradients and how do we find them?

*Functions of vectors and matrices: partial differentiation*

- For the kinds of functions (and programs) that we'll look at *optimising* in this course have a number of typical properties:
  - They are scalar-valued
    - \* We'll look at programs with *multiple losses*, but ultimately we can just consider optimising with respect to the *sum* of the losses.
  - They involve multiple variables, which are often wrapped up in the form of vectors or matrices, and more generally tensors.
  - **How will we find the gradients of these?**

## Recap: what are gradients and how do we find them?

*The chain rule for vectors*

Suppose that  $\mathbf{x} \in \mathbb{R}^m$ ,  $\mathbf{y} \in \mathbb{R}^n$ ,  $g$  maps from  $\mathbb{R}^m$  to  $\mathbb{R}^n$  and  $f$  maps from  $\mathbb{R}^n$  to  $\mathbb{R}$ .

If  $\mathbf{y} = g(\mathbf{x})$  and  $z = f(\mathbf{y})$ , then

$$\frac{\partial z}{\partial x_i} = \sum_j \frac{\partial z}{\partial y_j} \frac{\partial y_j}{\partial x_i}.$$

Equivalently, in vector notation:

$$\nabla_{\mathbf{x}} z = \left( \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \right)^{\top} \nabla_{\mathbf{y}} z$$

where  $\frac{\partial \mathbf{y}}{\partial \mathbf{x}}$  is the  $n \times m$  Jacobian matrix of  $g$ .

## Recap: what are gradients and how do we find them?

*Example:  $\nabla_{\mathbf{W}} f(\mathbf{XW})$*

- Let  $\mathbf{D} = \mathbf{XW}$  where the rows of  $\mathbf{X} \in \mathbb{R}^{n \times m}$  contain some fixed *features*, and  $\mathbf{W} \in \mathbb{R}^{m \times h}$  is a matrix of weights.
- Also let  $\mathcal{L} = f(\mathbf{D})$  be some scalar function of  $\mathbf{D}$  that we wish to minimise.
- What are the derivatives of  $\mathcal{L}$  with respect to the weights  $\mathbf{W}$ ?

## Recap: what are gradients and how do we find them?

*Example:  $\nabla_{\mathbf{W}} f(\mathbf{XW})$*

- Start by considering a specific weight,  $W_{uv}$ :  $\frac{\partial \mathcal{L}}{\partial W_{uv}} = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial D_{ij}} \frac{\partial D_{ij}}{\partial W_{uv}}$ .
- We know that  $\frac{\partial D_{ij}}{\partial W_{uv}} = 0$  if  $j \neq v$  because  $D_{ij}$  is the dot product of row  $i$  of  $\mathbf{X}$  and column  $j$  of  $\mathbf{W}$ .
- Therefore, we can simplify the summation to only consider cases where  $j = v$ :  $\sum_{i,j} \frac{\partial \mathcal{L}}{\partial D_{ij}} \frac{\partial D_{ij}}{\partial W_{uv}} = \sum_i \frac{\partial \mathcal{L}}{\partial D_{iv}} \frac{\partial D_{iv}}{\partial W_{uv}}$ .
- What is  $\frac{\partial D_{iv}}{\partial W_{uv}}$ ?

$$\begin{aligned} D_{iv} &= \sum_{k=1}^m X_{ik} W_{kv} \\ \frac{\partial D_{iv}}{\partial W_{uv}} &= \frac{\partial}{\partial W_{uv}} \sum_{k=1}^m X_{ik} W_{kv} = \sum_{k=1}^m \frac{\partial}{\partial W_{uv}} X_{ik} W_{kv} \\ &\therefore \frac{\partial D_{iv}}{\partial W_{uv}} = X_{iu} \end{aligned}$$

## Recap: what are gradients and how do we find them?

Example:  $\nabla_{\mathbf{W}} f(\mathbf{X}\mathbf{W})$

- Putting everything together, we have:  $\frac{\partial \mathcal{L}}{\partial W_{uv}} = \sum_i \frac{\partial \mathcal{L}}{\partial D_{iv}} X_{iu}$ .
- As we're summing over multiplications of scalars, we can change the order:  $\frac{\partial \mathcal{L}}{\partial W_{uv}} = \sum_i X_{iu} \frac{\partial \mathcal{L}}{\partial D_{iv}}$ .
- and note that the sum over  $i$  is doing a dot product with row  $u$  and column  $v$  if we transpose  $X_{iu}$  to  $X_{ui}^\top$ :  
 $\frac{\partial \mathcal{L}}{\partial W_{uv}} = \sum_i X_{ui}^\top \frac{\partial \mathcal{L}}{\partial D_{iv}}$ .
- We can then see that if we want this for all values of  $\mathbf{W}$  it simply generalises to:  $\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{X}^\top \frac{\partial \mathcal{L}}{\partial \mathbf{D}}$ .

## Recap: what are gradients and how do we find them?

*That worked, but the indices were a pain!*

- What did we actually do in the previous derivation?
  - We worked out which elements of  $\mathbf{D}$  depended on a particular element  $W_{uv}$ .
  - We multiplied each local derivative by the corresponding sensitivity  $\frac{\partial \mathcal{L}}{\partial D_{ij}}$ .
  - We summed over all of the ways in which changing  $W_{uv}$  could affect the loss.
- This is all perfectly manageable for  $\mathbf{D} = \mathbf{X}\mathbf{W}$ ...
  - ...but we don't want to repeat that index-heavy argument for every operation in a deep model!

## Recap: what are gradients and how do we find them?

*STOP! What does a gradient actually mean?*

- In your early calculus lessons you likely had it hammered into you that gradients represent rates of change of functions.
- This is of course totally true...
- But, it isn't a particularly useful way to think about the gradients of a loss with respect to the weights of a parameterised function.
  - **The gradient of the loss with respect to a parameter tells you how much the loss will change with a small perturbation to that parameter.**

## Jacobians as local linear maps

*Packaging up all of those partial derivatives*

- For a function  $f: \mathbb{R}^m \rightarrow \mathbb{R}^n$ , the Jacobian collects all of the partial derivatives into an  $n \times m$  matrix:

$$(J_f(\mathbf{x}))_{ji} = \frac{\partial f_j}{\partial x_i}.$$

- More usefully, the Jacobian describes the best linear approximation to  $f$  close to  $\mathbf{x}$  (c.f. the first-order Taylor Series expansion):

$$f(\mathbf{x} + \Delta \mathbf{x}) \approx f(\mathbf{x}) + J_f(\mathbf{x})\Delta \mathbf{x}.$$

- So, the Jacobian tells us how a small input perturbation produces a small output perturbation.

## Jacobians as local linear maps

*Do we actually want the whole Jacobian?*

- Suppose  $f$  has millions of inputs and millions of outputs.
  - Writing down  $J_f$  explicitly would require trillions of entries!
- Fortunately, most of the time we only need to answer one of two questions:

1. Given a particular perturbation of the inputs, what happens to the outputs?
  2. Given a particular sensitivity at the outputs, what are the corresponding sensitivities at the inputs?
- These are *Jacobian-vector products* and *vector-Jacobian products*.

### Jacobian-vector products

*Push a perturbation forwards*

- Let  $\mathbf{v} \in \mathbb{R}^m$  describe a direction in which we perturb the input to  $f$ .
- The corresponding first-order change in the output is the **Jacobian-vector product** (JVP):

$$\begin{aligned} \text{JVP}_f(\mathbf{x}, \mathbf{v}) &= J_f(\mathbf{x})\mathbf{v} \\ &= \left. \frac{d}{d\epsilon} f(\mathbf{x} + \epsilon\mathbf{v}) \right|_{\epsilon=0}. \end{aligned}$$

- A JVP maps one chosen input perturbation to its resulting output perturbation.
  - We can compute this product without ever constructing the complete Jacobian.

### Vector-Jacobian products

*Pull a sensitivity backwards*

- Now let  $\mathbf{u} \in \mathbb{R}^n$  describe how sensitive some later scalar value is to the outputs of  $f$ .
- The literal vector-Jacobian product is  $\mathbf{u}^\top J_f(\mathbf{x})$ . As we write sensitivities as column vectors, we use its transpose:

$$\text{VJP}_f(\mathbf{x}, \mathbf{u}) = J_f(\mathbf{x})^\top \mathbf{u}.$$

- Equivalently, this is the gradient of the scalar-valued function  $\mathbf{u}^\top f(\mathbf{x})$ :

$$J_f(\mathbf{x})^\top \mathbf{u} = \nabla_{\mathbf{x}} (\mathbf{u}^\top f(\mathbf{x})).$$

- A VJP takes sensitivities at the output and propagates them back to the input.

### JVPs and VJPs

*Returning to  $\mathbf{D} = \mathbf{X}\mathbf{W}$*

- Consider the function  $g(\mathbf{W}) = \mathbf{X}\mathbf{W}$ .
- If we perturb the weights by  $\Delta\mathbf{W}$ , the JVP pushes that perturbation forwards:

$$\Delta\mathbf{D} = \text{JVP}_g(\mathbf{W}, \Delta\mathbf{W}) = \mathbf{X}\Delta\mathbf{W}.$$

- If  $\mathbf{G} = \frac{\partial \mathcal{L}}{\partial \mathbf{D}}$  contains the sensitivities arriving from later in the computation, the VJP pulls them backwards:

$$\text{VJP}_g(\mathbf{W}, \mathbf{G}) = \mathbf{X}^\top \mathbf{G} = \frac{\partial \mathcal{L}}{\partial \mathbf{W}}.$$

- This is exactly the result we obtained from the element-wise derivation.

### JVPs and VJPs

*The chain rule is composition of products*

- If  $h = g \circ f$ , then a perturbation can be pushed forwards one operation at a time:

$$J_h(\mathbf{x})\mathbf{v} = J_g(f(\mathbf{x}))(J_f(\mathbf{x})\mathbf{v}).$$

- Likewise, a sensitivity can be pulled backwards one operation at a time:

$$J_h(\mathbf{x})^\top \mathbf{u} = J_f(\mathbf{x})^\top (J_g(f(\mathbf{x}))^\top \mathbf{u}).$$

- For a scalar loss, the final sensitivity is simply 1.
  - Backpropagation (in the coming lectures) is a sequence of VJPs, starting at the loss and working backwards through the computation.

## JVPs and VJPs

*What about matrices and higher-order tensors?*

- Conceptually, we could flatten every tensor into a vector, construct the corresponding Jacobian, perform the product and reshape the result.
- In practice, we preserve the natural tensor shapes and directly compute the required product.
  - We avoid explicitly constructing a potentially enormous Jacobian.
  - The result has the natural shape of the input for a VJP, or the output for a JVP.
- Later, we'll see how forward- and reverse-mode automatic differentiation compute these products through an arbitrary program.